

C++ 直播电商秒杀消费后台：项目技术说明与面试手册

1. 项目定位

本项目是一个面向工程实践的 C++17 秒杀消费后台。它不追求堆叠业务模块，而是围绕一条完整的高并发交易主线，展示后端开发中最重要的几个问题：热点库存如何原子扣减、入口流量如何削峰、消息重复如何幂等、数据库事务如何落账、失败如何补偿、服务如何恢复以及系统如何测试。

项目只关注：

- 用户注册、登录和 Session 鉴权；
- 商品和秒杀活动查询；
- 秒杀活动准入、用户去重和配额控制；
- Redis 或本地库存预留；
- 异步订单队列和消费者；
- MySQL 订单、库存镜像和 Outbox；
- Kafka/Redpanda 事件、重试和 DLQ；
- 本地恢复、监控、压测和故障切换。

不包含购物车、优惠券、支付、物流等与当前项目主线无关的业务域。

2. 设计目标

2.1 低延迟入口

HTTP 请求线程只承担协议解析、鉴权、限流、活动准入、库存预留和消息入队，不在秒杀入口同步执行慢速 MySQL 订单事务。

2.2 不超卖

库存检查和扣减必须是一个原子动作。Redis 路径使用 Lua；本地路径使用受互斥保护的库存状态机。库存状态满足：

```
available >= 0
reserved >= 0
sold >= 0
available + reserved + sold = initial_stock
```

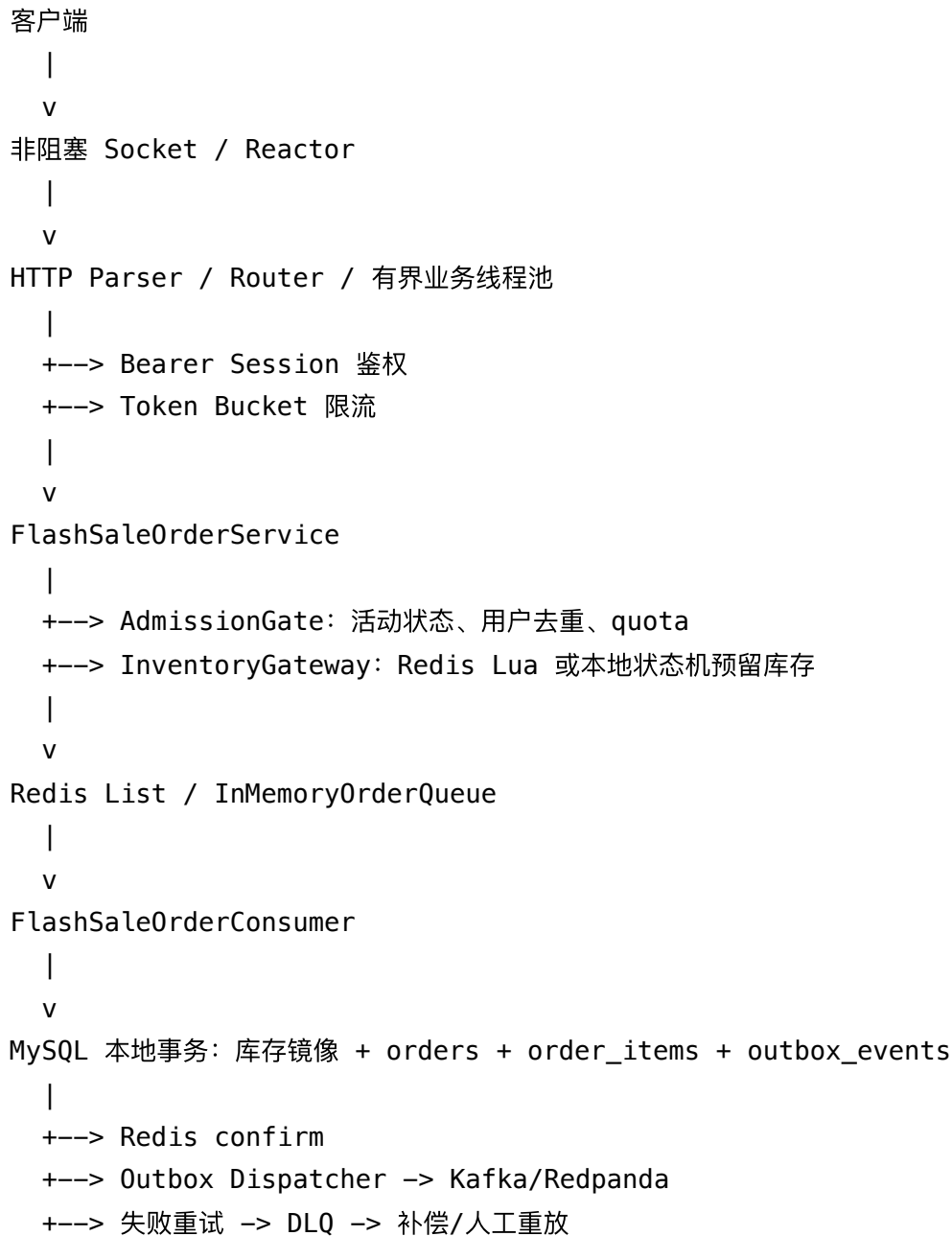
2.3 异步削峰

入口成功只返回“已排队”，订单由消费者异步处理。这样可以把突发请求和数据库写入解耦，使 MySQL 按可接受的速度消费消息。

2.4 最终一致和可恢复

Redis 负责秒杀入口的快速状态，MySQL 负责订单和交易账本。两者不做跨系统强事务，而是依赖本地事务、幂等、重试、DLQ、补偿和对账来收敛状态。

3. 运行主链路



入口返回 202 QUEUED 表示请求已经通过准入并成功入队，不表示订单已经完成。客户端通过订单查询接口确认最终状态。

4. 从启动到退出

4.1 进程启动

入口文件是 apps/ecommerce_api_main.cc 。启动过程如下：

1. 读取端口、线程数、队列容量、Redis、MySQL、Kafka、WAL、Snapshot 和 Outbox 的环境变量。
2. 创建本地存储、缓存、指标、日志、库存、活动、用户、订单和消息组件。
3. 如果配置了 Redis，建立 Redis KV、Redis 活动闸门、Redis Lua 库存和 Redis List 队列。
4. 如果配置了 MySQL，创建连接池并准备订单、库存和 Outbox 事务存储。
5. 如果配置了 Kafka/Redpanda，启动事件生产者和秒杀消费者。
6. 注册 HTTP 路由，启动 EventLoop、业务线程池和 Snapshot Worker。
7. 监听 TCP 端口，开始接收请求。

4.2 请求处理

HTTP Server 接收连接后，由 Buffer 累积数据并交给 HTTP Parser。Router 根据方法和路径分发请求，业务慢操作进入有界线程池。队列达到上限时快速失败，避免无限排队导致内存失控。

秒杀请求的大致处理顺序：

1. 解析 Authorization: Bearer <token> 。
2. 从 Session 中得到真实用户，不信任请求体内的 user_id 。
3. 检查 Token Bucket 是否允许继续处理。
4. 检查活动是否处于 RUNNING 状态。
5. 按活动和用户执行一次性准入。
6. 用 operation_id 原子预留库存。
7. 生成版本化订单消息并入队。
8. 入队失败时释放库存、回滚活动准入并返回错误。
9. 入队成功返回 202 和订单标识。

4.3 消费和落库

消费者取出消息后：

1. 校验消息版本、长度和必填字段。
2. 以订单 ID 或幂等键查询是否已经处理。
3. 在 MySQL 一个本地事务中完成库存镜像、订单、订单明细和 Outbox 写入。
4. 事务成功后确认 Redis 预留。
5. 临时错误重新消费；重复消息直接收敛为成功。
6. 达到最大重试次数后写入 DLQ，并释放或补偿入口状态。

4.4 优雅退出

收到 SIGINT 或 SIGTERM 后：

1. 停止接收新连接；
2. 停止接收新的业务任务；
3. 等待已有请求和消费者处理；
4. 刷新 Outbox、Kafka 和 Snapshot；
5. 关闭 Redis、MySQL、Kafka、Socket 和线程。

如果进程在中间崩溃，重启时通过 Snapshot 加载状态、重放 WAL、恢复 Outbox，并依靠消息重试和幂等继续处理未完成的任务。

5. 模块说明

5.1 apps：启动组装层

文件：apps/ecommerce_api_main.cc。

职责：读取环境变量、组装依赖和业务对象、根据配置选择 LocalKV 或 Redis、注册 API、启动和停止后台线程。它不实现库存扣减和订单规则，主要解决“如何把各个模块接成可运行服务”。

5.2 src/network：网络基础设施

主要文件：

- buffer.*：读写缓冲区、动态扩容、readv；
- reactor/event_loop.*：事件循环和 I/O 分发；
- reactor/channel.*：文件描述符事件注册；
- reactor/tcp_acceptor.*：监听和接受连接。

关键知识：非阻塞 Socket、Reactor、事件唤醒、半包、写缓冲、RAII、连接生命周期。网络层只负责 I/O，不负责 HTTP 业务和订单状态。

5.3 src/gateway：HTTP 接入层

主要文件：

- http/http_parser.*：请求行、Header、Body 和边界校验；
- http/http_response.*：响应构造；
- http/http_server.*：连接管理、线程池、Keep-Alive 和背压；
- router.*：方法和路径分发。

它负责协议适配和请求编排，不直接操作库存和 MySQL。典型接口包括：

```
GET /health
GET /metrics
GET /products/{sku}
GET /stock/{sku}
GET /promotions/{promotion}
POST /flash-sale/orders
GET /orders
```

5.4 src/business：核心业务域

promotion

负责活动生命周期：CREATED -> PREHEATING -> RUNNING -> FINISHED，同时负责资格、活动配额和用户去重。它是流量过滤层，不替代库存系统。

inventory

统一 reserve / confirm / release / rollback / query 接口，隐藏本地库存和 Redis 库存的差异。每次操作携带业务操作 ID，使重试不会重复改变库存。

flash_sale

负责秒杀入口编排和消费者处理：生成订单消息、入队、重试、补偿和 DLQ。

order

负责订单服务接口、订单查询和幂等边界。

product

负责 SKU 元数据和商品查询，查询路径可通过缓存和 KV 回源。

user

负责注册、登录、密码哈希、Session 和账号状态。生产化仍需接入更完整的权限和 TLS 体系。

5.5 src/cache：热点读缓存

实现文件：sharded_cache.*。采用分片 LRU、TTL 和 Singleflight：分片降低全局锁竞争，TTL 控制商品和活动数据的新鲜度，Singleflight 合并相同 Key 的并发回源。缓存只服务读路径，不作为订单最终事实源。

5.6 src/kv : KV 和 Redis 快速路径

主要能力：

- IKVStore 和 KVClient 抽象；
- Redis Get/Set/Del/CAS/Expire；
- RedisFlashSaleGate 活动准入和用户去重；
- RedisInventoryFastPath Lua 原子库存状态机；
- RedisOrderQueue 的 LPUSH/BRPOP 队列。

Redis Lua 把“检查状态、检查重复、检查库存、修改库存、记录操作 ID”放进一个原子脚本，避免并发请求在读写之间互相穿插。Redis 是秒杀入口快速状态，不是最终订单账本。

5.7 src/storage : 持久化和恢复

LocalKV

文件位于 storage/local_engine、storage/wal、storage/snapshot。写入顺序是 Append WAL -> fsync -> Apply -> ACK；启动顺序是 Load Snapshot -> Replay WAL。WAL 和 Snapshot 使用 CRC，损坏记录会被拒绝，避免静默加载错误状态。

MySQL

文件位于 storage/mysql，数据库结构在 storage/mysql/schema.sql。订单、库存镜像、订单明细和 Outbox 在一个 MySQL 本地事务中提交。订单幂等键和 Outbox 事件 ID 使用唯一约束防止重复落账。

5.8 src/messaging : 异步消息和可靠投递

主要组件：

- InMemoryOrderQueue：本地测试队列；
- RedisOrderQueue：秒杀主队列；
- FlashSaleOrderMessage：版本化、长度分隔的消息；
- TransactionalOutbox、FileOutbox：可靠事件暂存；
- KafkaProducer/KafkaConsumer：Kafka/Redpanda 适配；
- KafkaFlashSaleOrderConsumer：消费、重试、DLQ、补偿；
- KafkaDlqOperator：DLQ 重投。

投递语义是至少一次，不宣称消息系统天然提供恰好一次。系统依靠业务幂等键、数据库唯一键和条件更新达到业务上的重复收敛。

5.9 src/traffic : 入口限流

TokenBucket 使用“按时间补充 Token、按请求消耗 Token”的方式限流：

```
tokens = min(capacity, tokens + elapsed * rate)
```

Token 不足时快速拒绝。当前实现是进程内限流，多实例之间不是全局共享额度。

5.10 src/observability : 可观测性

包括结构化日志和指标注册表，提供请求数量和延迟、限流拒绝、队列深度、库存预留成功/失败、Outbox 待发布数量、消费和 DLQ 统计，以及 /health 和 /metrics 。

6. 测试体系

6.1 tests/ : 证明正确性

- first_stage_test : HTTP 边界、缓存、限流、用户恢复和 Outbox 基础行为；
- storage_recovery_test : WAL/Snapshot CRC、批量恢复、文件锁和孤儿预占释放；
- commerce_concurrency_test : 多线程库存预留，成功数不能超过库存；
- security_outbox_test : Session 注销、Outbox 重启和重试；
- flash_sale_pipeline_test : 准入、库存、入队、消费者和入队失败补偿；
- mysql_transaction_test : 真实 MySQL 事务、订单幂等、库存镜像和 Outbox；
- redis_concurrency_test : 真实 Redis Lua 并发扣减；
- external_smoke_test : Redis、Kafka 基础连通和业务操作；
- kafka_flash_sale_test : 消费者组、重试、DLQ 和库存补偿。

6.2 tools/ : 测量性能和容量

- http_load_generator : 单接口吞吐、P50/P95/P99 和最大延迟；
- http_business_load_generator : 健康、商品、库存、活动和指标只读混合访问；
- flash_sale_soak_test : 本地秒杀流水线长时间稳定性；
- mysql_capacity_test : MySQL 订单事务吞吐和尾延迟；
- redis_concurrency_test : 真实 Redis 库存并发能力；
- external_smoke_test 、 kafka_flash_sale_test : 外部依赖和消息链路验证。

工具的结果不能混淆：/health 的短连接 RPS 不等于秒杀成交 QPS，本地内存长稳也不等于完整 Redis/MySQL/Kafka 生产容量。

6.3 常用执行命令

普通测试：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo -DENABLE_MYSQL=ON
cmake --build build --parallel
ctest --test-dir build --output-on-failure
```

Sanitizer：

```
./deploy/scripts/test.sh
RUN_TSAN=1 ./deploy/scripts/test.sh
```

真实依赖：

```
docker compose -f deploy/compose/docker-compose.yml up -d redis mysql redpanda
cmake -S . -B build-external -DENABLE_MYSQL=ON -DENABLE_EXTERNAL_TESTS=ON
cmake --build build-external --parallel
ctest --test-dir build-external --output-on-failure
```

长稳和 HTTP：

```
FLASH_SOAK_SECONDS=60 ./build-external/flash_sale_soak_test
./build-external/http_load_generator 127.0.0.1 8080 /health 10000 64
./build-external/http_business_load_generator 127.0.0.1 8080 10000 64
```

7. deploy/：可复现交付

- docker/Dockerfile：多阶段构建，分离编译环境和运行环境；
- compose/docker-compose.yml：单 API + Redis + MySQL + Redpanda；
- compose/docker-compose.multi.yml：两个 API 实例共享外部依赖；
- scripts/build.sh：统一构建；
- scripts/test.sh：普通测试和 Sanitizer；
- scripts/long_soak.sh：长稳入口；
- scripts/multi_instance_failover.sh：双实例基本故障切换。

多实例 Compose 能证明 Session、Redis 库存和 MySQL 订单可以跨 API 实例共享，但依赖服务仍是单节点；它不是 Redis Cluster、MySQL 主从或跨机房高可用证明。

8. 一小时面试讲法

0~10 分钟：讲目标

面试官：这个项目解决什么问题？

候选人：这是一个 C++ 秒杀消费后台。入口优先保证低延迟和不超卖，把突发请求转成 Redis 原子预留和异步消息；消费者再使用 MySQL 本地事务完成订单账本。重点不是业务模块数量，而是高并发、异步化、一致性和故障恢复。

10~25 分钟：讲链路

面试官：一次秒杀请求如何完成？

候选人：请求进入 Reactor 后由 HTTP 层解析，经过 Bearer Session 和 Token Bucket，活动闸门完成状态、用户去重和 quota 判断，Redis Lua 原子预留库存，成功后把版本化消息放入 Redis List，返回 202。消费者按幂等键写 MySQL 订单、库存镜像和 Outbox，提交后确认 Redis 预留。

25~40 分钟：讲一致性

面试官：Redis 扣库存后 MySQL 失败怎么办？

候选人：Redis 预留不是最终成交。消息消费失败会重试；超过阈值进入 DLQ，并执行释放或补偿。MySQL 订单事务使用唯一幂等键，重复消息不会重复落账。Redis 和 MySQL 的极端差异交给对账任务处理。

面试官：为什么不使用 Redis 和 MySQL 分布式事务？

候选人：跨系统强事务会增加锁和网络等待，不适合秒杀入口低延迟。这里把问题拆成 Redis 原子预留、消息至少一次、MySQL 本地事务、幂等和可补偿状态机，工程上更容易扩展和恢复。

40~52 分钟：讲并发和 C++

面试官：如何证明不超卖？

候选人：Redis 中把检查和扣减放在同一个 Lua 脚本，本地实现用互斥保护库存状态机；测试用远大于库存的并发请求，验证成功数、库存不变量和重复操作结果。HTTP I/O 线程不做慢数据库事务，业务线程池有界并且队列满时快速失败。

面试官：为什么用至少一次而不是恰好一次？

候选人：网络和进程故障下很难同时保证消息和数据库只执行一次。至少一次可以重试，业务层通过唯一键、幂等查询和条件更新把重复执行收敛为一次有效业务结果。

52~60 分钟：讲测试和边界

面试官：现在能证明什么？

候选人：可以证明本地和真实 Redis/MySQL/Kafka 路径的主要正确性，包括防超卖、订单幂等、Outbox 恢复、消息重试、DLQ 和补偿；还提供 HTTP、Redis、MySQL 容量和长稳工具。当前不能把单机短连接压测结果包装成生产集群最大 QPS，真实多机故障切换、依赖集群容量和大规模数据库容量仍需独立环境验证。

9. 当前工程边界和后续优化

当前明确边界：

- LocalKV 只适合零依赖单机运行，不能作为多实例共享事实源；
- Token Bucket 是进程内状态，多实例需要 Redis 分布式限流；
- Compose 依赖服务是单节点，不代表高可用集群；
- HTTP 工具是基础压测工具，不是完整生产流量回放平台；
- DLQ 已具备基础重投能力，但告警、审计和人工运维闭环仍需生产系统补齐。

合理的后续顺序是：先补齐真实订单写路径的长时间容量数据，再引入统一分布式限流和对账告警，最后才考虑 Redis Cluster、数据库代理、分片和多机故障演练。不要在没有基准数据前继续堆叠业务模块。

10. 模块到文件的快速索引

模块	入口目录或文件	关键词
启动	apps/ecommerce_api_main.cc	依赖组装、路由、信号
网络	src/network	Reactor、Buffer、EventLoop
网关	src/gateway	HTTP、Router、线程池
业务	src/business	活动、库存、秒杀订单、用户
缓存	src/cache	分片 LRU、TTL、Singleflight
KV	src/kv	Redis、Lua、List、CAS

模块	入口目录或文件	关键词
存储	src/storage	WAL、Snapshot、MySQL 事务
消息	src/messaging	Outbox、Kafka、DLQ
限流	src/traffic	Token Bucket
观测	src/observability	Metrics、结构化日志
测试	tests	正确性、恢复、外部依赖
工具	tools 、 benchmarks	压测、容量、长稳
部署	deploy	Docker、Compose、故障切换

11. 按大厂后端面试要求逐模块深挖

前面的章节用于快速建立全局认识。本章从面试官的视角继续拆解每个模块：它解决什么问题、代码如何实现、为什么这样实现、哪里可能出故障、面试官会怎么追问。

11.1 启动模块： apps/ecommerce_api_main.cc

模块要解决的问题

业务类不能自己决定 Redis、MySQL、Kafka 和线程如何创建，否则业务代码会和具体基础设施强耦合。启动模块是 Composition Root，负责把接口和实现组装起来。

实际实现

启动代码按照以下顺序建立对象：

```
配置读取
-> LocalKV / Redis
-> LocalInventory / RedisInventoryGateway
-> LocalAdmissionGate / RedisFlashSaleGate
-> LocalQueue / RedisOrderQueue
-> LocalRepository / MySqlTransactionStore
-> Consumer / Outbox Dispatcher
-> HttpServer 和路由
```

通过 `#ifdef ECOMMERCE_HAS_REDIS`、`ECOMMERCE_HAS_MYSQL` 和 `ECOMMERCE_HAS_KAFKA`，编译时决定第三方适配器是否可用；通过 `ECOMMERCE_INVENTORY_MODE` 和 `ECOMMERCE_FLASH_SALE_QUEUE`，运行时选择本地或 Redis 路径。

这种设计让同一套业务测试可以使用内存实现，真实集成测试再替换为 Redis、MySQL 和 Kafka 实现。

信号和生命周期

`SIGINT`、`SIGTERM` 只设置关闭标志并唤醒主循环，不在信号处理函数中执行复杂逻辑。真正的关闭动作由主线程负责，依次停止接入、停止工作线程、等待任务、刷新 Outbox/Snapshot，再释放外部连接。

面试追问

面试官：为什么不把 Redis 连接写在 `FlashSaleOrderService` 里面？

回答：业务层依赖 `IInventoryGateway`、`IAdmissionGate` 和 `IOrderQueue` 等稳定接口，不应该感知 hiredis、MySQL C API 或 Kafka 配置。这样本地测试可以替换 Fake 实现，线上再替换外部适配器，也能避免业务逻辑和基础设施初始化互相污染。

面试官：如果 Redis 配置存在但连接失败，服务应该降级到本地模式吗？

回答：要区分开发和生产。开发模式可以显式允许 LocalKV 回退；生产模式不能静默回退，因为本地库存不是共享事实源，多实例下会产生超卖和状态分裂。当前项目通过显式模式控制，生产部署应对关键依赖失败快速失败并告警。

11.2 网络模块： `src/network`

模块要解决的问题

高并发连接下，不能为每个连接创建一个阻塞线程。网络模块使用非阻塞文件描述符和事件循环，在少量线程中管理大量连接。

EventLoop 的实现

EventLoop 内部维护：

- poller 文件描述符：Linux 使用 `epoll`，macOS 使用 `poll`；
- `channels_`：文件描述符到 `Channel` 的映射；
- wakeup fd：Linux 使用 `eventfd`，macOS 使用 `pipe`；
- `pending_functors_`：其他线程提交给 I/O 线程执行的回调。

主循环逻辑是：

等待 I/O 事件

- > 处理连接可读/可写/错误事件
- > 处理 wakeup
- > 执行 pending callbacks
- > 继续等待

`runInLoop` 如果调用者已经在 `EventLoop` 线程中，就直接执行；否则把回调放入队列并写 `wakeup fd`，保证 `EventLoop` 不会一直阻塞在 `epoll_wait` 或 `poll`。

Buffer 的实现

Buffer 用 `reader_index_` 和 `writer_index_` 管理可读区、可写区和 `prepend` 区。读数据时优先写入现有空间，空间不足时再扩容或搬移可读数据；`readFd` 使用 `readv`，一次系统调用同时写入主缓冲区和临时栈缓冲区。

它解决了三个问题：

1. TCP 没有消息边界，HTTP 请求可能分多次到达；
2. 一次读取可能包含多个请求，需要保留未消费字节；
3. Socket 不应该因为暂时写不完而阻塞线程。

线程模型

网络线程只做 I/O 就绪、协议解析和响应回写；可能阻塞的业务任务提交给 Gateway 的有界线程池。跨线程响应通过 `EventLoop` 的唤醒机制回到 I/O 线程。

面试追问

面试官：为什么需要 `wakeup fd`？

回答：业务线程完成任务后需要通知 I/O 线程回写响应或更新连接状态。如果只把回调放在内存队列里，I/O 线程可能还阻塞在 `poll`，无法及时执行；`wakeup fd` 能把等待中的事件循环立即唤醒。

面试官：ET 和 LT 有什么区别？项目为什么使用 LT？

回答：LT 只要 `fd` 仍有数据就会持续通知，处理不完整时不容易丢事件，适合先保证正确性；ET 只有状态变化时通知，吞吐可能更高，但必须循环读到 `EAGAIN`，否则会因为没有再次通知而卡住。当前项目优先使用 LT，后续可以单独做 ET 基准。

面试官：网络线程中能不能直接访问 MySQL？

回答：不能。MySQL 查询、密码哈希和外部 Redis 调用都可能阻塞 I/O 线程。应该把慢操作放入有界业务线程池，完成后再通过 EventLoop 回写。

11.3 网关模块： src/gateway

HTTP Parser 的边界设计

解析器不假设一次 read 就拿到完整请求，而是区分三种结果：

- Ok：已经拿到一个完整请求；
- Incomplete：需要继续从 Socket 读取；
- InvalidArgument：请求格式错误，应返回 4xx 或关闭连接。

当前解析器具备以下边界：

- 只接受 HTTP/1.1；
- 要求 Host；
- 限制 Header 总大小和 Body 大小；
- 严格解析 Content-Length；
- 拒绝重复 Header；
- 当前不接受非 identity 的 Transfer-Encoding；
- 返回 consumed，保留 Keep-Alive 或流水线请求的剩余字节。

这比简单的字符串 find 更接近真实 HTTP 服务，因为畸形请求、半包、粘包和超大 Body 都属于生产中的输入边界。

Router 和线程池

Router 负责方法、路径和 Handler 映射；Handler 负责把 HTTP 参数转换成业务请求，不在 Router 中写库存状态迁移。

ThreadPool 使用固定数量工作线程和有界 deque。提交任务时如果队列达到容量就返回失败，由 Gateway 快速返回过载错误。这样形成背压：

流量过大

- > 入口限流
- > 有界线程池
- > 队列满时拒绝
- > 保护进程内存和外部依赖

面试追问

面试官：为什么线程池必须有界？

回答：无界队列会把过载请求全部积存在内存中，表面上成功率高，实际会造成延迟无限增长、内存耗尽，并把压力继续传给 MySQL。秒杀系统更重要的是可控拒绝和稳定尾延迟。

面试官：Keep-Alive 如何处理半包和多请求？

回答：连接拥有一个持续 Buffer，每次追加 Socket 数据；Parser 返回第一个完整请求消耗的字节数，处理后只移除已消费部分，剩余字节继续解析。遇到 Incomplete 就继续读，遇到非法请求按协议返回错误或关闭连接。

11.4 业务模块： src/business

活动状态机

活动不是一个简单的布尔值，而是有限状态机：

CREATED -> PREHEATING -> RUNNING -> FINISHED

预热阶段准备活动配置、库存和用户准入结构；运行阶段允许用户抢购；结束阶段拒绝新请求。状态转移必须校验前置状态，避免活动被重复启动或从 finished 回到 running。

秒杀入口编排

FlashSaleOrderService::submit 的核心顺序是：

参数校验

```
-> admission.tryAcquire  
-> inventory.reserve  
-> queue.push
```

三个动作不是一个跨系统事务，因此需要补偿：

- 准入成功、库存失败：回滚准入；
- 库存成功、入队失败：释放库存并回滚准入；
- 入队成功：返回 queued，由消费者负责后续流程。

这个顺序把昂贵数据库事务移出请求入口，同时把每个失败边界显式表达出来。

库存状态机

库存不是直接从 available 减到 sold，而是：

```
available --reserve--> reserved --confirm--> sold
reserved  --release--> available
```

每个 reservation 携带 business_operation_id。重复 reserve、confirm 或 release 都应该返回 Duplicate 或已完成状态，而不是再次修改数值。

用户和订单

用户模块负责 Token Session 和密码哈希；订单模块负责订单查询和交易存储接口。用户身份来自认证 Session，不能信任请求体里的 user_id。订单幂等键由客户端或入口生成，并在消息和数据库中贯穿整个链路。

面试追问

面试官：为什么先扣 Redis 再写 MySQL？

回答：Redis Lua 更适合承受秒杀入口的热点并发，MySQL 负责最终订单账本。入口的目标是快速过滤和预留，消费者再按照数据库吞吐异步落账。代价是需要消息、幂等、补偿和对账，这些能力在项目中显式实现。

面试官：入队失败为什么不能简单返回错误？

回答：库存和活动准入已经发生了状态变化。如果只返回错误而不补偿，用户不能再次购买，库存也会被永久占用。因此必须释放 reservation、回滚 admission，并记录指标和日志。

11.5 缓存模块： src/cache

Sharded LRU

缓存由多个 Shard 组成，每个 Shard 有自己的互斥锁、LRU 链表、Key 到 Entry 的哈希表和 Singleflight 表。Key 通过哈希映射到固定分片，避免所有请求竞争一把全局锁。

一次命中操作是：

定位 shard

- > 加 shard 锁
- > 查 unordered_map
- > 判断 TTL
- > 命中则移动到 LRU 链表头部

容量超过上限时淘汰链表尾部。过期数据采用访问时惰性删除，避免额外的全局定时扫描线程。

Singleflight

getOrLoad 先尝试缓存；未命中后，第一请求创建 promise/shared_future 并执行 loader，后续相同 Key 的请求复用同一个 future。这样热点 Key 失效时只发起一次回源查询。

它解决的是缓存击穿，不解决缓存穿透和缓存雪崩。生产化还需要：

- 对不存在 Key 做短 TTL 空值缓存；
- TTL 加随机抖动，避免同一时间大量失效；
- 对回源失败设置退避和熔断；
- 对热点 Key 做预热或多级缓存。

面试追问

面试官：为什么缓存命中还要移动 LRU 链表？

回答：LRU 的定义是最近最少使用。命中后必须把 Key 放到链表头部，否则淘汰顺序无法反映真实访问热度。

面试官：Singleflight 的 future 没有完成会怎样？

回答：loader 必须保证最终设置 promise；生产实现还要捕获异常、设置超时并在异常路径清理 inflight，否则后续请求会永久等待。当前项目的边界测试覆盖正常成功和失败路径，异常安全仍应作为进一步增强点。

11.6 KV 模块： src/kv

抽象层

IKVStore / KVClient 把 Get、Set、Del、CAS、Expire 等语义从业务中抽离。业务层关注“用户 Session”“活动闸门”“库存操作”，而不是 hiredis 的连接和命令格式。

Redis 活动准入

RedisFlashSaleGate 用两个 Key：活动元数据和用户集合。Lua 脚本一次完成：

检查活动状态

- > 检查用户是否已参与
- > 检查 `admitted < quota`
- > 写入用户集合
- > `admitted + 1`

用户去重和 quota 更新在一个 Redis 原子执行单元内，因此不会出现两个请求同时看到剩余名额并都通过的问题。

Redis 库存 Lua

库存 Hash 保存 `total/available/reserved/sold`，操作 Hash 保存 operation ID 和状态。Reserve、Confirm、Release、Rollback 都由 Lua 执行，返回数字状态码，再映射为 C++ 枚举。

Key 使用相同的 hash tag，例如 `flash_inventory:{sku}:stock` 和 `flash_inventory:{sku}:operations`，为未来 Redis Cluster 的同槽执行留下接口边界。

Redis List 队列

生产者使用 LPUSH，消费者使用 BRPOP。BRPOP 是阻塞命令，所以实现单独的 Redis consumer connection，避免阻塞普通请求使用的连接。

面试追问

面试官：为什么用 Lua 而不是 GET 后再 DECR？

回答：GET、判断、DECR 是多个命令，中间会被其他请求插入，可能超卖。Lua 在 Redis 内部原子执行，能把检查和变更放在一个执行单元中。

面试官：为什么不直接使用 Redis 事务 MULTI/EXEC？

回答：MULTI/EXEC 能批量提交，但复杂条件判断和分支处理不如 Lua 直接；Lua 还可以在服务端读取值、判断状态并一次完成多键变更。若使用 Cluster，多 Key 还必须保证同一 hash slot。

11.7 存储模块：src/storage

WAL 和 Snapshot

本地路径使用 WAL 保证写入先落日志，再应用到内存；Snapshot 定期保存压缩后的完整状态。

写入: WAL append -> fsync -> apply -> ack
启动: load snapshot -> replay WAL -> service ready

WAL 记录带长度、Magic 和 CRC。最后一条因进程崩溃而不完整的记录可以识别；中间位置的完整记录损坏则拒绝启动，避免静默丢数据。

Snapshot Worker 负责周期性创建快照和截断已经覆盖的 WAL。它是单进程恢复能力，不是多实例共享数据库。

MySQL 连接池

MySQLConnectionPool 维护空闲连接和全部连接， Lease RAI 对象在析构时归还连接。acquire 使用条件变量等待可用连接并带超时，避免连接池耗尽时无限阻塞。

MySQL 订单事务

消费者通过 MySQLTransactionStore::persistQueuedOrder 执行本地事务：

```
BEGIN
-> 按幂等键查询已有订单
-> 条件更新库存镜像
-> INSERT orders
-> INSERT order_items
-> INSERT outbox_events
COMMIT
```

任一步失败就 rollback。唯一索引和条件更新是数据库层最后一道并发保护，不能只依赖 C++ 代码中的判断。

面试追问

面试官：数据库事务提交成功但 Redis confirm 失败怎么办？

回答：消息不会被当作完全成功，消费者可以重试 confirm；confirm 本身按 operation ID 幂等。若 Redis 长时间不可用，事件进入 DLQ 或待处理状态，再由对账任务比较 MySQL 账本和 Redis 快速状态。MySQL 订单不能因为 Redis 短暂失败而重复创建。

面试官：连接池解决什么问题？

回答：连接建立有握手和认证成本，不能每个订单重新连接；连接池复用连接，并通过池大小限制数据库并发。池太小会排队，池太大又会打爆 MySQL，应该结合事务耗时、数据库连接上限和 P99 压测确定。

11.8 消息模块： `src/messaging`

消息格式

`FlashSaleOrderMessage` 使用 Magic、版本号、长度分隔字段和数量字段，而不是简单拼接字符串。这样可以防止分隔符注入、限制字段长度，并让未来消费者识别不兼容版本。

至少一次消费

Kafka Consumer 只有在业务 Handler 返回成功后才提交 offset。进程在数据库提交后、offset 提交前崩溃，会导致消息重复，这是预期行为；数据库唯一键和幂等查询负责消除重复影响。

Redis List 路径的 BRPOP 会在取出时移除消息，可靠性主要依赖消费者重试、DLQ 和补偿。若要生产化增强，可以引入 processing list、可见性超时和显式 ACK，或统一使用 Kafka 作为可重放队列。

Outbox

业务事务先写 Outbox，再由 Dispatcher 发布到 Kafka/下游。FileOutbox 采用长度 framing、CRC、fsync 和重写未发布记录，进程重启后可以恢复 pending events。发布成功和标记完成之间仍可能崩溃，因此下游必须按 event_id 幂等。

DLQ 和补偿

消息达到最大重试次数后写入 DLQ，消费者释放库存和回滚准入。DLQ 不是“丢弃消息”，而是把无法自动处理的消息转为可观测、可重放的状态。

面试追问

面试官：如何保证 Kafka 消息不丢？

回答：消费者处理成功之前不提交 offset，失败就让消息重新可见；业务结果通过 MySQL 事务持久化，重复消费由唯一键和幂等处理收敛。严格来说是至少一次，不是消息层恰好一次。

面试官：Outbox 为什么还需要下游幂等？

回答：发布成功后进程可能在更新 published 标记前崩溃，重启会再次发布同一事件。Outbox 解决的是“事件不丢”，不能天然消除“事件重复”，所以消费者必须按 event ID 幂等。

11.9 限流模块： `src/traffic`

Token Bucket

桶有容量 C 、当前 Token 数量 T 、速率 r 和上次补充时间 t_0 。请求到达时：

```
T = min(C, T + (now - t0) * r)
如果 T >= request_cost: 通过并扣减
否则: 拒绝
```

实现用互斥保护浮点 Token 和时间点，保证多线程调用不会同时消费同一批 Token。

为什么限流要放在入口

越靠近入口拒绝，越能节省线程池、Redis 和 MySQL 资源。实际链路可以组合：

- 网关连接上限
 - > 用户/接口限流
 - > 活动 quota
 - > 库存原子预留
 - > 队列容量背压

当前 Token Bucket 是单进程状态，多实例部署时每个实例都有自己的桶。若要求全局额度，应使用 Redis Lua 或独立限流服务；同时需要考虑 Redis 限流依赖失败时的 fail-open/fail-close 策略。

11.10 观测模块： src/observability

指标应该回答什么问题

指标不是为了“有数据”，而是对应故障定位问题：

问题	指标
请求是否变慢	请求计数、P50/P95/P99、EventLoop 延迟
是否过载	活跃连接、线程池队列深度、限流拒绝
是否库存异常	reserve 成功/失败、confirm/release、库存不变量
是否消费堆积	队列深度、消费数量、失败和 DLQ 数量
是否事件积压	Outbox pending、发布成功/失败

结构化日志至少要带 request ID、方法、路径、状态码、耗时和错误原因，使单次请求能够关联到后续的消息和订单。

当前项目实现了基础 Metrics、日志、健康检查和指标接口，但没有把完整分布式 Trace、Prometheus/Grafana 告警规则作为主线能力。面试时应明确区分已实现能力和生产化演进方向。

11.11 测试模块： tests

测试分层

单元测试：函数、边界、不变量

-> 集成测试：多个模块协作

-> 恢复测试：崩溃、损坏、重启

-> 外部依赖测试：真实 Redis/MySQL/Kafka

正确性测试的核心不是覆盖率数字，而是验证不变量：

- 成功预留数不超过初始库存；
- available/reserved/sold 永远非负；
- 库存总量守恒；
- 同一个用户同一个活动不能重复成功；
- 同一个幂等键最多一笔订单；
- 重复消息不会重复扣减；
- 入队失败和毒消息会释放预留。

Sanitizer 的作用

- ASAN：堆越界、Use-after-free、重复释放；
- UBSAN：整数溢出、非法转换和未定义行为；
- TSAN：线程间数据竞争。

Sanitizer 不是压力测试的替代品：它提高错误发现能力，但会改变性能和时序，应该和普通 RelWithDebInfo 压测分开。

外部依赖测试的判断

外部测试必须记录服务版本、机器规格、连接池大小、库存规模、并发、持续时间、成功率和 P95/P99。只看到测试程序退出码为 0，不代表测到了系统上限；需要同时观察 Redis 延迟、MySQL 锁等待、Kafka lag、CPU、内存和网络。

11.12 工具和基准模块： tools 、 benchmarks

工具和结论的对应关系

工具	只能证明什么	不能直接证明什么
http_load_generator	某个 HTTP 路径的吞吐和尾延迟	完整秒杀成交能力
http_business_load_generator	多个只读接口的混合读性能	写链路和 MySQL 消费容量
redis_concurrency_test	Redis Lua 库存并发正确性	API 集群总 QPS
mysql_capacity_test	订单事务路径容量	Redis/Kafka/HTTP 全链路容量
flash_sale_soak_test	本地流水线长时间稳定性	真实依赖故障切换能力
inventory_benchmark	本地库存状态机基线	真实用户并发数

压测报告必须写清测试合同：请求比例、并发模型、数据量、持续时间、SLO、成功率定义和终止条件。平均延迟不能替代 P99，短连接 RPS 不能直接写成生产 QPS。

11.13 部署模块： deploy

Docker

Dockerfile 使用多阶段构建：构建阶段安装编译器和开发库，运行阶段只保留可执行文件和运行库。这样可以缩小运行镜像，并避免把编译工具带入运行环境。

Compose

单实例 Compose 提供：

```
gateway + Redis + MySQL + Redpanda
```

多实例 Compose 提供两个 API 实例，共享 Redis、MySQL 和 Redpanda，用于验证 Session、库存和订单数据跨实例共享，以及单个 API 实例停止后的基本可用性。

故障切换边界

当前脚本只验证 API 层故障切换，不代表依赖服务高可用。Redis、MySQL、Redpanda 仍是单节点，缺少副本选主、数据同步、连接代理和跨机房演练。面试时主动说明边界，比声称“支持高可用”更可

信。

12. 面试官可能继续追问的系统性问题

问题一：为什么入口返回 202，而不是同步返回订单成功？

因为入口只完成准入、预留和入队，订单写库尚未完成。返回 202 能准确表达异步状态；如果返回 200/成功，客户端会误以为订单已经持久化。

问题二：活动 quota 和库存数量为什么都要控制？

quota 是活动参与资格的流量闸门，库存是 SKU 的交易资源。它们可能不同：一个活动可以限制参与人数，也可能一个用户购买多件。两个状态机分离能避免把资格控制和实际库存混成一个难以恢复的变量。

问题三：为什么订单 ID 不能单独作为幂等依据？

订单 ID 适合识别一次业务订单，但客户端重试可能携带不同订单 ID。幂等键要由业务请求贯穿入口、消息和数据库，数据库对它建立唯一约束；订单 ID 可以作为操作 ID，但不能替代所有业务幂等语义。

问题四：消费者在什么时刻确认 Redis reservation？

应在 MySQL 订单事务成功提交之后确认。提前确认可能导致 MySQL 失败但 Redis 已经变成 sold；太晚则会让 reserved 长时间堆积。确认操作必须幂等，失败时通过重试和对账处理。

问题五：如何防止库存补偿把正常库存加回去？

补偿必须带 operation ID，并检查当前 reservation 状态。只有仍然是 reserved 的操作才允许 release；已经 confirmed 或 released 的重复补偿返回 Duplicate/Conflict，不再次修改库存。

问题六：MySQL 订单事务中的库存更新为什么要用条件更新？

即使入口已经在 Redis 扣过库存，MySQL 仍然是最终账本，数据库更新必须防止并发和重复消费绕过业务层。条件更新可以把库存充足、状态正确作为 SQL 层原子条件，降低应用层竞态风险。

问题七：如果队列积压怎么办？

先通过 queue depth、consumer lag、MySQL latency 判断瓶颈；入口侧降低准入或限流，消费者侧增加实例/线程但不能超过 MySQL 承载能力；对长期失败消息转入 DLQ，不能无限重试拖垮正常消息。

问题八：为什么不直接把所有请求写入 MySQL？

热点 SKU 会让大量请求竞争同一行库存，连接、锁和事务日志成为入口瓶颈。Redis Lua 先做快速原子过滤，只有成功预留的少量消息进入数据库写路径，达到削峰和保护数据库的目的。

问题九：单机模式有什么价值？

单机模式让开发者可以在没有 Redis、MySQL、Kafka 的情况下验证业务状态机、WAL、Snapshot、线程和协议边界；接口抽象保证切换到真实依赖时不修改核心业务。它不能作为多实例事实源，但很适合作为快速回归环境。

问题十：项目目前离生产系统还差什么？

需要进一步补齐统一分布式限流、Redis/MySQL 对账调度、DLQ 告警和人工重放审计、真实依赖集群、数据库代理和读写策略、全链路 Trace、压测流量模型以及多机故障注入。这些属于生产化能力，不应伪装成当前已经完成。

13. 推荐学习和讲解顺序

如果要真正掌握项目，建议按照以下顺序阅读：

1. 先读本文件第 1~4 章，能不看代码画出完整主链路；
2. 阅读 `apps/ecommerce_api_main.cc`，理解所有实现如何组装；
3. 阅读 `gateway` -> `business/flash_sale` -> `business/inventory` -> `kv`，掌握一次请求的正向路径；
4. 阅读 `messaging` -> `storage/mysql`，掌握异步消费、事务、幂等和失败补偿；
5. 阅读 `storage/local_engine` -> `wal` -> `snapshot`，掌握崩溃恢复；
6. 阅读 `network` 和 `cache`，补足 Reactor、线程池、锁竞争和缓存击穿知识；
7. 阅读 `tests`，把每个设计不变量和测试用例对应起来；
8. 阅读 `tools`、`benchmarks`、`deploy`，理解性能数据的边界和运行环境；
9. 最后用第 8 章和第 12 章进行一小时模拟面试。

达到能面试的标准不是背诵类名，而是能够回答四个问题：请求为什么这样流转、并发下为什么不超卖、失败后如何恢复、测试如何证明结论。只要这四个问题能结合具体文件和代码讲清楚，项目就具备大厂后端实习项目应有的技术深度。

14. 各模块独立讲义

下面按照“定位、文件、实现、运行、测试、面试和边界”的固定结构，对每个模块进行独立拆解。阅读时可以把每一节当作一个面试主题准备。

14.1 apps：服务启动与依赖组装

一、模块定位

apps 是应用启动层，不负责表达具体业务规则，也不直接实现 Redis Lua、MySQL SQL 或 Kafka 消费逻辑。它负责把配置、网络层、业务层和基础设施适配器组装成一个可以运行的进程。

它解决的问题是：

- 业务接口很多
- 外部依赖很多
- 本地和真实运行模式不同
- 如何在一个地方完成统一组装？

当前入口文件：

```
apps/ecommerce_api_main.cc
```

二、依赖组装关系

启动代码最终形成以下对象图：

EventLoop

- > HttpServer
 - > Router
 - > UserService
 - > ProductService
 - > PromotionService
 - > FlashSaleOrderService
 - > OrderService

FlashSaleOrderService

- > IAdmissionGate
- > IInventoryGateway
- > IOrderQueue

FlashSaleOrderConsumer

- > IFlashSaleOrderRepository
- > IInventoryGateway
- > IAdmissionGate

接口左边是业务依赖，具体实现由启动层决定：

IAdmissionGate

- > FlashSaleAdmissionGate
- > RedisFlashSaleGate

IInventoryGateway

- > InventoryService 适配器
- > RedisInventoryGateway

IOrderQueue

- > InMemoryOrderQueue
- > RedisOrderQueue

IFlashSaleOrderRepository

- > LocalFlashSaleOrderRepository
- > MySqlTransactionStore

三、启动顺序

读取环境变量

- > 初始化本地存储和恢复状态
- > 连接 Redis/MySQL/Kafka
- > 初始化库存和活动
- > 构造业务服务
- > 构造消费者和 Outbox Dispatcher
- > 注册 HTTP 路由
- > 启动 EventLoop 和 worker
- > 开始监听端口

外部依赖连接失败时必须区分依赖是否关键。例如 Redis 模式要求 Redis 可用，不能静默切回单机本地库存；否则两个 API 实例会各自维护一份库存，产生超卖。

四、信号和优雅退出

进程捕获 SIGINT 和 SIGTERM，先停止接收新请求，再停止消费者和线程池，等待正在执行的任务，最后刷新 Outbox/Snapshot 并关闭外部连接。复杂清理不放在 signal handler 中，避免在异步信号上下文中调用非异步安全函数。

五、面试追问

面试官：为什么入口文件可以依赖所有模块，而业务模块不能依赖具体实现？

回答：入口文件本身就是组合根，知道整个应用需要哪些组件；业务模块只应该依赖抽象接口，保证领域逻辑能被单元测试和不同存储实现复用。这是依赖倒置和可测试性的基础。

面试官：本地模式和 Redis 模式共用一套业务代码，靠什么切换？

回答：靠 IAdmissionGate、IInventoryGateway、IOrderQueue 和 Repository 接口切换具体实现，启动层根据环境变量和编译宏选择实现，业务服务不感知底层连接细节。

六、当前边界

当前启动层是单进程应用入口，不包含服务注册中心、动态配置中心、灰度发布、统一 Secret 管理和完整健康探针体系。Docker Compose 能复现本地环境，但不是生产部署平台。

14.2 src/network: Reactor、Buffer 和 EventLoop

一、模块定位

src/network 是网络基础设施层，负责：

- 非阻塞 Socket；
- TCP 监听和接受连接；
- 文件描述符事件注册；
- EventLoop 事件分发；
- 跨线程唤醒；
- Buffer 读写和动态扩容。

它不负责 HTTP 语义、路由和秒杀业务。

主要文件：

```
buffer.*  
reactor/channel.*  
reactor/event_loop.*  
reactor/tcp_acceptor.*
```

二、连接建立流程

```
TcpAcceptor listen  
-> accept 非阻塞连接  
-> 创建 Connection  
-> 创建读事件 Channel  
-> 注册到 EventLoop  
-> readable 时读取 Buffer
```

当 Socket 可读时，EventLoop 调用 Channel 的回调；Buffer 先保存字节流，Gateway 再判断是否形成完整 HTTP 请求。

三、EventLoop 的实现

EventLoop 的核心状态包括：

```
int poller_fd_;
int wakeup_read_fd_;
int wakeup_write_fd_;
unordered_map<int, Channel*> channels_;
vector<Functor> pending_functors_;
```

Linux 使用 epoll，macOS 使用 poll；Linux 使用 eventfd 唤醒，macOS 使用 pipe。这样同一套接口可以在开发机和 Linux 服务器上运行。

主循环逻辑：

```
epoll_wait / poll
-> 找到就绪 fd
-> Channel 设置 revents
-> Channel::handleEvent()
-> 处理 wakeup
-> 执行 pending functor
```

runInLoop 解决了其他线程想修改连接状态或回写响应的问题：业务线程只把回调提交回 I/O 线程，由 I/O 线程统一操作连接对象。

四、Buffer 的实现

Buffer 用 reader_index 和 writer_index 管理可读区、可写区和 prepend 区：

```
[ prependable | readable | writable ]
      ^ reader      ^ writer
```

retrieve 移动读指针；retrieveAll 重置读写指针；makeSpace 在容量不足时选择扩容或搬移可读数据。readFd 使用 readv，减少一次读取时的系统调用次数。

TCP 只保证字节顺序，不保证消息边界，所以 Buffer 必须支持：

- 半个请求先到；
- 一个请求分多次到；
- 一次读到多个请求；
- 请求后仍有下一个请求的剩余字节。

五、线程模型和生命周期

I/O 线程：accept、read、parse、write、connection state

业务线程：认证、缓存回源、Redis/MySQL、秒杀编排

网络层不允许 I/O 线程直接等待 MySQL。连接对象通过 shared_ptr/weak_ptr 和关闭状态控制异步回调生命周期，避免业务任务结束时访问已经析构的连接。

六、面试追问

面试官：为什么需要 EventLoop 唤醒？

回答：业务线程完成任务后要让 I/O 线程及时回写响应。如果不唤醒，I/O 线程可能还阻塞在 `epoll_wait`；`wakeup fd` 能把它立即唤醒。

面试官：如果 write 一次没有写完怎么办？

回答：不能在 I/O 线程中循环阻塞写完，而是保留未写完数据到输出 Buffer，注册可写事件，下一次 `writable` 再继续发送。生产中还需要高水位和慢客户端保护。

七、测试对应关系

`first_stage_test` 验证 EventLoop wakeup 和 HTTP 边界；HTTP 压测工具验证连接接入、解析和响应回写；Sanitizer 用于检查连接和异步回调生命周期问题。

八、当前边界

当前实现优先保证跨 macOS/Linux 的基础正确性，Linux 生产性能还需要专门的 `epoll/ET`、连接复用、零拷贝和多 EventLoop benchmark。当前网络层不是完整的 HTTP/2、TLS 或生产级连接治理框架。

14.3 src/gateway：HTTP、Router 和线程池

一、模块定位

`src/gateway` 是客户端接入层，负责：

- HTTP/1.1 请求解析；
- Header、Body 和请求边界校验；
- Method/Path 路由；
- Bearer Token 提取；
- Keep-Alive；
- 业务线程池调度；

- 响应生成和连接关闭。

主要文件：

```
http/http_parser.*
http/http_response.*
http/http_server.*
router.*
```

二、HTTP 解析流程

Buffer 累积字节

- > 查找 CRLF CRLF
- > 解析 request line
- > 解析 Header
- > 校验 Host、Content-Length、Transfer-Encoding
- > 等待完整 Body
- > 返回 consumed 字节数

解析器返回三类状态：

Ok	一个完整请求已形成
Incomplete	数据不足，需要继续读取
Invalid	请求非法，应拒绝

当前限制 Header 和 Body 大小，拒绝重复 Header 和非 identity Transfer-Encoding，避免请求走私和无限增长。

三、Router 的边界

Router 只根据请求方法和路径找到 Handler，Handler 负责把请求转成业务 DTO。库存扣减、订单状态迁移和消息重试不能写进 Router，否则协议层和业务层会强耦合。

核心路由：

```
GET /health
GET /metrics
GET /products/{sku}
GET /stock/{sku}
GET /promotions/{promotion}
POST /flash-sale/orders
GET /orders
```

四、有界线程池

common::ThreadPool 维护固定 worker 和有限任务队列：

```
submit
-> stopping? reject
-> queue full? reject
-> push task
-> notify worker
```

队列满时快速失败是背压的一部分：

```
限流挡住部分请求
-> 线程池处理有限请求
-> 队列满时拒绝
-> 不让内存和 MySQL 无限堆积
```

五、鉴权 and 用户身份

需要身份的接口从 Authorization: Bearer 获取用户。请求体中的 user_id 只作为不可信输入，最终以 Session 查出的用户为准。注销、禁用用户和 Token 失效都由 UserService 处理。

六、面试追问

面试官：为什么解析器要返回 consumed？

回答：Keep-Alive 连接上的一次读取可能包含多个 HTTP 请求。返回第一个完整请求消耗的字节数后，Buffer 可以删除已消费部分并保留后续请求，不需要复制全部剩余数据。

面试官：为什么不支持 Chunked？

回答：当前秒杀接口请求体很小且采用 Content-Length，先限制协议复杂度，避免未实现的 Transfer-Encoding 造成边界问题。生产化可以专门实现 Chunked 状态机并补充安全测试。

七、测试对应关系

单元测试覆盖请求行、Header、Content-Length、重复 Header、半包和超大 Body；HTTP 工具观察接口吞吐和尾延迟；长连接、慢客户端和连接异常属于后续压力专项。

14.4 src/business：秒杀业务领域层

一、模块定位

src/business 只表达业务规则，不直接处理 Socket、HTTP 解析、Redis 连接、MySQL SQL、Kafka 通信和线程事件循环。它通过接口依赖外部能力，由 apps/ecommerce_api_main.cc 组装实现。

当前业务子模块：

user	用户注册、登录、Session、鉴权
product	秒杀商品信息
promotion	活动生命周期、资格和 quota
inventory	预留、确认、释放和对账
flash_sale	秒杀入口、消费者、重试和补偿
order	订单模型、幂等落库和查询

二、业务主链路

用户请求

- > user 鉴权
- > promotion 活动准入
- > inventory 原子预留
- > flash_sale 写入队列
- > consumer 消费
- > order repository 持久化
- > inventory confirm
- > Outbox/Kafka 事件发布

三、user：用户和鉴权

用户模型包含 ID、用户名、密码哈希、角色和 enabled 状态。密码使用随机盐和 PBKDF2-HMAC-SHA256，不保存明文，也不使用 std::hash 作为密码方案。

Session 有两条路径：

LocalKV: 单进程内存和 WAL

Redis: 共享 session token 到 user_id 的映射

Redis 模式下, 第二个 API 实例可以识别第一个实例创建的 Token, 实现 API 层的无状态化; 但这不等于完整权限中心和 Refresh Token 体系。

四、product: 商品信息

商品模块维护 SKU、名称、价格和上架状态。商品查询可使用 ShardedCache 和 KV 回源, 但不负责扣库存。商品信息和库存状态分开, 避免商品缓存更新影响交易库存。

五、promotion: 活动准入

活动状态:

CREATED -> PREHEATING -> RUNNING -> FINISHED

IAdmissionGate 抽象本地实现和 Redis 实现。tryAcquire 要在一个原子语义中完成状态判断、用户去重、quota 判断、用户集合写入和 admitted 增加。

六、inventory: 库存状态机

库存流转:

```
available --reserve--> reserved --confirm--> sold
reserved  --release--> available
```

每个操作带 business_operation_id, 使重复 reserve、confirm、release 都能够识别为 Duplicate 或 Conflict, 而不是重复修改数值。

七、flash_sale: 同步入口和异步消费者

同步入口 submit 只做参数检查、准入、库存预留、消息构造和入队。消费者执行订单持久化和库存确认。入口返回 202 是因为订单尚未完成 MySQL 提交。

八、order: 订单和幂等

Local 模式使用 LocalFlashSaleOrderRepository, MySQL 模式使用 MySqlTransactionStore。消费者依赖统一的 IFlashSaleOrderRepository, 订单幂等由 order ID、idempotency key、数据库唯一键共同保证。

九、业务失败补偿

准入成功、库存失败

-> rollback admission

库存成功、入队失败

-> release inventory

-> rollback admission

消费者多次失败

-> DLQ

-> release/compensate

十、面试总结

可以这样回答：

src/business 是秒杀后台的领域层，负责活动、库存、秒杀订单、用户和订单规则，通过 Gate、InventoryGateway、Queue 和 Repository 接口隔离基础设施。

入口用 Redis 原子预留和异步入队降低延迟，消费者用幂等事务落 MySQL，并用重试、DLQ 和补偿处理异常。

十一、当前边界

当前订单模型只覆盖秒杀创建和查询，不扩展支付、取消、物流和履约状态。MySQL 已能承载核心事务，但 Redis/MySQL 极端不一致仍需要独立的生产级对账调度和告警闭环。

14.5 src/cache：分片 LRU、TTL 和 Singleflight

一、模块定位

src/cache 负责商品、活动等高频读数据的本地热点缓存，不负责订单最终事实和库存最终一致性。

文件：

src/cache/sharded_cache.h

src/cache/sharded_cache.cc

二、数据结构

每个 Shard 包含：

```
mutex
list<string> lru
unordered_map<Key, Entry + iterator> entries
unordered_map<Key, shared_future<Result>> inflight
```

Map 负责 O(1) 查找，List 负责 O(1) 移动和淘汰。Entry 保存 Value 和过期时间点。

三、访问流程

```
key hash -> shard
-> lock shard
-> lookup map
-> expired? 删除并 miss
-> hit? 移到 LRU 头部并返回
```

容量超过每分片限制时淘汰尾部 Key。TTL 当前采用访问时惰性过期，不需要独立清理线程。

四、Singleflight 流程

```
getOrLoad
-> 先查缓存
-> miss
-> inflight 已存在: 等待 shared_future
-> 不存在: 创建 promise, 当前请求执行 loader
-> loader 成功: set cache + set promise
-> 删除 inflight
```

这能把同一个热点 Key 的并发回源合并成一个，但不同 Key 之间仍然可以并行。

五、缓存问题与解决方案

问题	当前能力	生产增强
缓存击穿	Singleflight	已有基础方案
缓存穿透	由业务 loader 控制	空值缓存、参数校验
缓存雪崩	TTL	随机 TTL、分批预热
热点 Key	分片和本地缓存	多级缓存、热点隔离
数据更新	TTL 回源	事件失效或主动更新

六、面试追问

面试官：为什么要分片，而不是给整个缓存一把锁？

回答：全局锁会让不相关 Key 的请求互相阻塞，热点读下锁竞争会成为瓶颈。分片后不同 Key 落在不同锁上，降低竞争；代价是容量和统计需要按分片管理。

面试官：Singleflight 的 future 没有完成会怎样？

回答：loader 必须保证最终设置 promise；生产实现还要捕获异常、设置超时并在异常路径清理 inflight，否则后续请求会永久等待。当前项目覆盖正常成功和失败路径，异常安全仍可继续增强。

14.6 src/kv：Redis、Lua、List 和 CAS

一、模块定位

src/kv 隔离业务与 KV 实现，承担两类能力：

通用 KV：Get/Set/Del/CAS/Expire

秒杀快速路径：活动准入、库存 Lua、Redis List 队列

二、RedisFlashSaleGate

活动元数据保存状态、quota 和 admitted，用户参与记录保存到 Set。Lua 脚本把以下动作合为原子单元：

```
state 是否 running
-> SISMEMBER 用户集合
-> admitted < quota
-> SADD 用户
-> HINCRBY admitted
```

返回数字码再映射成业务枚举，避免 Redis 细节泄露到业务层。

三、RedisInventoryFastPath

库存 Hash：

```
flash_inventory:{sku}:stock
  total
  available
  reserved
  sold
```

操作 Hash:

```
flash_inventory:{sku}:operations
  operation_id -> quantity|reserved/confirmed/released
```

Reserve 脚本先查操作 ID，再查 available，最后一次性修改 available/reserved 并记录状态。Confirm 和 Release 也验证当前状态，避免非法逆向修改。

四、Redis Cluster 兼容考虑

Key 使用 {sku} hash tag，使同一 SKU 的库存 Hash 和操作 Hash 落到同一槽位。当前项目使用单 Redis，hash tag 是为未来多分片执行 Lua 留下的设计约束，不代表当前已经部署 Redis Cluster。

五、RedisOrderQueue

生产者 LPUSH 序列化消息，消费者 BRPOP 并反序列化。BRPOP 会阻塞连接，所以构造专用 blocking connection，不能和普通 Redis 请求混用。

六、CAS 和脚本加载

通用 KV 的 CAS 用于并发条件更新；Lua 执行优先使用 SCRIPT LOAD/EVALSHA，遇到 NOSCRIPT 时重新加载脚本。这样减少重复传输脚本的开销，同时处理 Redis 重启或脚本缓存丢失。

七、面试追问

面试官：Redis Lua 执行期间其他命令能不能插入？

回答：脚本在 Redis 中串行执行，其他命令不会在脚本内部插入，因此同一个脚本内的检查和修改具有原子性。但脚本过长会阻塞 Redis，生产中要控制脚本复杂度和执行时间。

面试官：为什么库存操作一定要记录 operation ID？

回答：消息重试和客户端重试都会带来重复操作。operation ID 是状态机的幂等索引，可以把重复 reserve/confirm/release 收敛为不重复变更。

14.7 src/storage: WAL、Snapshot 和 MySQL 事务

一、模块定位

src/storage 包含两条持久化路径：

LocalKV/WAL/Snapshot：零依赖单机运行和恢复

MySQL：订单、库存镜像、订单明细和 Outbox 最终账本

二者职责不同，不能把 LocalKV 当成多实例共享数据库。

二、LocalKV 写入

生成批量记录

- > 写 WAL
- > flush/fsync
- > 应用到内存 Map
- > 返回成功

如果先更新内存再写 WAL，进程在两步之间崩溃，重启后就会丢失已经返回成功的数据。因此日志先行。

三、启动恢复

读取 Snapshot

- > 校验 Magic/长度/CRC
- > 加载内存
- > 顺序重放 WAL
- > 恢复服务

中间完整记录损坏时拒绝启动；末尾不完整记录可以按崩溃尾部处理。这样避免把损坏状态当作正确状态运行。

四、Snapshot Worker

Snapshot Worker 周期性生成快照并压缩已经覆盖的 WAL，优雅退出时再尝试执行最终快照。快照不是替代 WAL，而是缩短重启时需要重放的日志长度。

五、MySQL 连接池

连接池用空闲连接队列和条件变量控制并发，Lease 采用 RAII 自动归还。获取连接有超时，避免数据库不可用时所有业务线程无限等待。

六、MySQL 订单事务

```
BEGIN
-> 幂等查询
-> 条件扣减 MySQL 库存镜像
-> INSERT orders
-> INSERT order_items
-> INSERT outbox_events
COMMIT
```

任一步失败就 rollback。orders.idempotency_key 和 outbox_events.event_id 的唯一约束是数据库层幂等保证。

七、面试追问

面试官：为什么不把 Redis 库存和 MySQL 订单放进一个事务？

回答：Redis 和 MySQL 是两个独立系统，强行做分布式事务会增加协调、锁和网络延迟，秒杀入口不适合。项目选择 Redis 原子预留 + 消息 + MySQL 本地事务 + 幂等补偿，把不可避免的不一致控制在可恢复范围内。

面试官：数据库连接池大小越大越好吗？

回答：不是。连接池大小受 MySQL 最大连接数、事务平均耗时、锁竞争和 CPU 限制。过小会让业务线程排队，过大则把压力集中打到数据库，必须通过容量测试确定。

14.8 src/messaging：Outbox、Kafka 和 DLQ

一、模块定位

src/messaging 负责订单消息、异步队列、事件投递和失败处理，不负责定义商品或库存业务规则。

主要文件：

```
order_message.*
order_queue.*
in_memory_broker.*
outbox.h
async_outbox_dispatcher.*
kafka_client.*
kafka_order_queue.*
kafka_flash_sale_consumer.*
kafka_dlq_operator.*
```

二、消息格式

订单消息采用 Magic、Version、长度分隔字段和固定数值字段：

```
magic
version
promotion_id
order_id
idempotency_key
user_id
sku_id
quantity
```

反序列化时限制总长度、字段长度和版本，畸形消息不会直接进入业务处理。

三、消费者处理

```
pop/poll
-> deserialize/validate
-> persistQueuedOrder
-> inventory.confirm
-> success/commit
```

Kafka 只有 Handler 返回成功后才提交 offset。Redis List 当前是取出即删除，失败消息由重试、DLQ 和补偿兜底；生产化可以增强为可靠队列或统一使用 Kafka。

四、Outbox 处理

业务事务

- > 写订单和 Outbox
- > Dispatcher drain
- > Kafka publish
- > 标记/删除已发布事件

FileOutbox 对记录使用长度和 CRC，追加时 fsync，重启时恢复 pending 事件。发布和标记之间如果崩溃，会重复发布，所以事件消费者还需要 event ID 幂等。

五、重试和 DLQ

临时失败

- > retry

达到 max_attempts

- > publish DLQ
- > release inventory
- > rollback admission

DLQ 的价值是把异常消息隔离，避免一条毒消息阻塞所有正常消息，同时保留后续分析、修复和重放的可能。

六、面试追问

面试官：消息重复和消息丢失哪个更容易解决？

回答：重复通常可以通过唯一键和幂等消费解决；丢失更危险，需要在生产确认、数据库提交、Outbox 和消费位点之间建立持久化边界。项目选择至少一次投递，宁可重复也不静默丢失。

面试官：Outbox 为什么还需要下游幂等？

回答：发布成功后进程可能在更新 published 标记前崩溃，重启会再次发布同一事件。Outbox 解决的是“事件不丢”，不能天然消除“事件重复”，所以消费者必须按 event ID 幂等。

14.9 src/traffic：Token Bucket 限流

一、模块定位

src/traffic 只保留当前主链路实际使用的 Token Bucket，负责秒杀入口的进程内快速限流。线程池背压、消息重试和数据库连接池不放在这个模块重复实现。

文件：

src/traffic/token_bucket.h

二、算法

参数：

rate_per_second：补充速度

burst：桶容量

tokens：当前 Token

last_refill：上次补充时间

每次获取 Token 时先根据经过时间补充，再判断是否足够。互斥保证多个请求不会同时消费同一个 Token。

三、为什么使用 Token Bucket

相比固定窗口，Token Bucket 能允许一定突发流量，同时长期平均速率受控：

正常流量：平滑通过

短时突发：消耗 burst

持续过载：Token 耗尽后快速拒绝

四、面试追问

面试官：Token Bucket 放在多实例部署会有什么问题？

回答：当前每个进程有独立桶，多个实例总额度会被放大。若需要全局限流，应把 Token 状态放入 Redis Lua 或独立限流服务，但要设计依赖故障时的 fail-open/fail-close 和本地兜底。

14.10 src/observability：Metrics 和结构化日志

一、模块定位

可观测性模块回答三个问题：

服务是否活着？

服务是否变慢？

请求和消息失败在哪里？

主要文件：

```
metrics/metrics_registry.*  
logging/structured_logger.*
```

二、指标分类

请求: request_total、status、latency
网络: active_connections、event_loop_lag
线程: worker queue depth、rejected tasks
业务: reserve success/fail、order idempotent hit
消息: queue depth、processed、retry、DLQ
存储: outbox pending、WAL fsync latency

指标需要与故障动作对应。例如 queue depth 持续增加时，说明消费者处理速度低于入口生产速度；此时不能只继续增加 API 实例，而要检查 MySQL 事务和消费者容量。

三、结构化日志

错误日志至少要包含：

```
request_id  
method/path  
status_code  
elapsed_us  
business_operation_id  
order_id  
dependency/status
```

这样可以从 HTTP 请求追到库存操作、订单消息和数据库事务。

四、健康检查和指标接口

GET /health
-> 进程和基础运行状态

GET /metrics
-> 请求、延迟、队列、库存、Outbox 等指标

健康检查适合给容器编排做存活判断；业务依赖是否完全可用不应简单混在 liveness 中，否则 Redis 短暂故障可能导致容器不断重启。

五、面试追问

面试官：为什么只打印日志不够？

回答：日志适合定位单次异常，但无法高效观察趋势和分位延迟。指标适合看整体速率、错误率、队列积压和 P99；生产系统还需要 Trace 把跨服务调用串起来。

六、当前边界

当前实现是轻量级 Metrics 和结构化日志，没有完整 Prometheus/Grafana、Trace、告警路由和日志集中存储。面试时应把它定位为工程基础能力，不包装成完整可观测平台。

14.11 tests：正确性、恢复和外部依赖

一、模块定位

tests 不是简单的“跑几个样例”，而是验证秒杀系统的核心不变量和故障语义。

目录：

```
tests/unit
tests/integration
```

二、单元测试

first_stage_test 快速验证 HTTP、EventLoop、缓存、LocalKV、密码、Outbox 和限流等局部行为。它应该在每次代码修改后优先执行。

三、并发和流水线集成测试

flash_sale_pipeline_test 验证准入、库存、消息、消费者和失败补偿；commerce_concurrency_test 用远大于库存的请求验证不超卖；security_outbox_test 验证 Session 和 Outbox 重启。

四、恢复测试

storage_recovery_test 人为构造 WAL/Snapshot CRC 损坏、单写者冲突和孤儿 reservation，验证系统不会静默加载错误状态，恢复后不会永久占用库存。

五、真实外部依赖测试

`redis_concurrency_test`

-> 真实 Redis Lua 并发库存

`mysql_transaction_test`

-> 真实 MySQL 订单事务和幂等

`kafka_flash_sale_test`

-> 真实 Kafka/Redpanda 消费、重试、DLQ

`external_smoke_test`

-> Redis/Kafka 连通和关键命令

外部依赖测试需要在 CMake 中开启 `ENABLE_EXTERNAL_TESTS`，并先启动 Redis、MySQL 和 Redpanda。缺少依赖时不能把“测试跳过”解释为“依赖链路通过”。

六、测试不变量

成功数 <= 初始库存

`available + reserved + sold = initial_stock`

同一用户/活动最多一次准入

同一幂等键最多一笔订单

重复消息不重复扣库存

入队失败会补偿

毒消息进入 DLQ

七、Sanitizer

ASAN -> 堆越界、Use-after-free、重复释放

UBSAN -> 整数溢出、非法转换、未定义行为

TSAN -> 数据竞争和线程同步问题

Sanitizer 不是压力测试的替代品：它提高错误发现能力，但会改变性能和时序，应该和普通 RelWithDebInfo 压测分开。

八、面试追问

面试官：并发测试只看程序退出码够不够？

回答：不够。还要检查业务不变量、成功数、最终库存、订单数量、消息数量、错误分布以及依赖指标。一个没有崩溃但超卖的程序仍然是失败的。

14.12 tools 和 benchmarks：压测、容量和长稳

一、模块定位

tools 面向重复执行的外部依赖和接口实验，benchmarks 面向某个核心组件的基准。它们的结果必须写明测试对象，不能把局部指标包装成全链路能力。

二、HTTP 工具

http_load_generator 测单路径吞吐和 P50/P95/P99；http_business_load_generator 混合健康、商品、库存、活动和指标读接口。

它们使用 POSIX Socket 建立请求，当前更适合单机基线。短连接 /health 的高 RPS 只能说明网络接入和简单 Handler 的能力。

三、Redis 和 MySQL 工具

redis_concurrency_test 验证真实 Lua 库存原子性；mysql_capacity_test 验证订单事务的吞吐和尾延迟。二者分别测不同瓶颈，不能相互替代。

四、长稳和库存基准

flash_sale_soak_test 使用本地流水线长时间提交和消费，检查队列最终清空、库存守恒和 processed/queued 一致；inventory_benchmark 测本地库存预留的并发基线。

五、正确的压测记录

每次正式压测都应记录：

- 机器 CPU/内存
- 编译类型和 Sanitizer
- 依赖服务版本
- 数据规模和库存量
- 并发模型、总请求、持续时间
- 成功率、吞吐、P50/P95/P99、最大延迟
- CPU、内存、连接池、队列深度

六、面试追问

面试官：你能说项目最大 QPS 是多少吗？

回答：必须先说明测试合同。当前工具能提供单机 HTTP、Redis 库存、MySQL 事务和本地长稳样本，但不能把其中任一值直接称为生产全链路最大 QPS。完整结论还需要真实依赖、明确读写比例、SLO 和故障注入。

14.13 deploy: Docker、Compose 和故障切换

一、模块定位

deploy 让项目可以被重复构建、启动、测试和演练。它不是业务代码，而是工程交付层。

目录：

```
docker/Dockerfile
compose/docker-compose.yml
compose/docker-compose.multi.yml
scripts/build.sh
scripts/test.sh
scripts/long_soak.sh
scripts/multi_instance_failover.sh
```

二、Dockerfile

多阶段构建：

```
builder image
-> 安装编译器和开发库
-> CMake build

runtime image
-> 只安装运行库
-> 复制 ecommerce_api_demo
-> 暴露 8080
```

运行镜像不携带编译器和源码，降低镜像体积和运行环境复杂度。

三、单实例 Compose

```
gateway:8080
redis:6379
mysql:3306
redpanda:9092
```

API 使用 Redis 处理 Session、活动、库存和订单队列，MySQL 保存订单和 Outbox，Redpanda 承担后续事件。

四、多实例 Compose

```
gateway-1:8080
gateway-2:8081
共享 Redis/MySQL/Redpanda
```

它能验证共享 Session、共享库存和 API 实例故障后的基本可用性，但依赖服务仍是单节点；LocalKV 文件也必须是每个实例自己的卷，不能把本地 WAL 当作共享存储。

五、故障切换脚本

```
启动两个 API
-> 等待 /health
-> 在 gateway-1 注册并获得 Token
-> 在 gateway-2 使用 Token
-> 停止 gateway-1
-> 检查 gateway-2 仍可用
```

这证明的是 API 层共享状态和基础切换，不证明 Redis、MySQL、Redpanda 自动选主和数据恢复。

六、面试追问

面试官：Docker Compose 和 Kubernetes 的区别是什么？

回答：Compose 适合本地多容器编排和可复现实验，Kubernetes 负责生产级调度、探针、滚动发布、服务发现、资源隔离和副本管理。当前项目使用 Compose 是为了降低本地复现成本，不宣称已经完成生产集群交付。

面试官：为什么测试脚本里要有 ASAN/UBSAN/TSAN？

回答：普通测试验证业务结果，ASAN/UBSAN/TSAN 分别补充内存错误、未定义行为和数据竞争检测；它们改变运行时开销，不能和性能基准混在同一次实验中。

15. 全部模块串起来的一次面试白板

面试时建议先画下面这张图，再逐层展开：

```
Client
|
v
network: non-blocking Socket / EventLoop / Buffer
|
v
gateway: HTTP Parser / Router / bounded ThreadPool
|
+--> user: Session authentication
+--> traffic: Token Bucket
|
v
business: FlashSaleOrderService
|
+--> promotion: running + user dedup + quota
+--> kv: Redis Lua reserve
|
v
messaging: Redis List / Kafka message
|
v
business: FlashSaleOrderConsumer
|
+--> storage: MySQL local transaction
|         orders + order_items + inventory + outbox
|
+--> kv: Redis confirm
+--> messaging: retry / DLQ / Outbox publish
|
v
observability: metrics + structured logs
tests/tools/deploy: prove, measure and reproduce
```

面试官无论从网络、C++ 并发、Redis、MySQL 还是消息队列切入，最终都应该能回到同一条主线：入口快速过滤，库存原子预留，消息异步削峰，数据库事务落账，重复可幂等，失败可补偿，状态可恢复，结果可观测。